High-order finite elements | SFEM

Exploring cellwise layouts for matrix-free FE operators

Preliminary work

Gabriele Marchi and Patrick Zulian

Università della Svizzera Italiana | 21 October 2026



The model problem

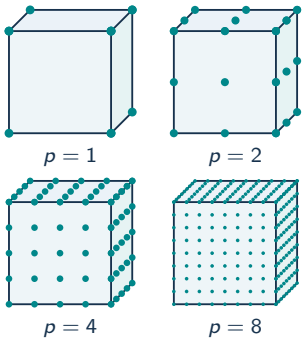
Poisson problem

Find $u = 0$ on $\partial\Omega$ such that

$$\int_\Omega \nabla v \cdot \nabla u \, \mathrm{d}x = \int_\Omega v f \, \mathrm{d}x.$$

Finite-element discretization

$$Au = b$$



Affine tensor-product Q_p hexahedra: $\phi_{ijk} = \ell_i(\xi)\ell_j(\eta)\ell_k(\zeta)$, with $(p+1)^3$ nodal values per element.

$p + 1$ Gauss points per direction integrate the affine-element stiffness exactly.

Classical global matrix-free apply

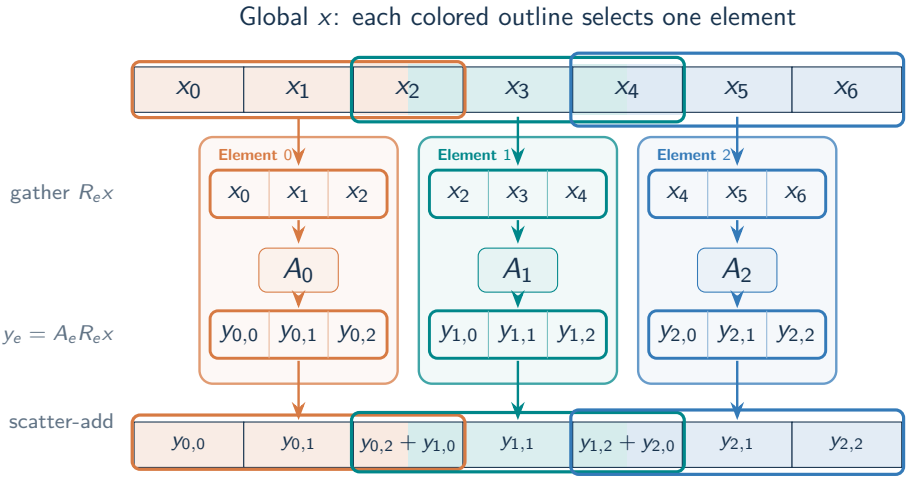
$$y = Ax = \sum_e R_e^{\mathsf{T}} A_e R_e x$$

R_e selects the global values belonging to element e.



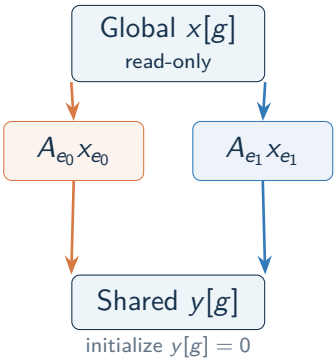
Focus of this work: how different memory layouts shape data traffic and access patterns around the local operator, especially gather and scatter-add.

Global gather, local apply, global scatter-add



One-dimensional Q_2 schematic; the hexahedral operator uses $(p+1)^3$ local values per element.

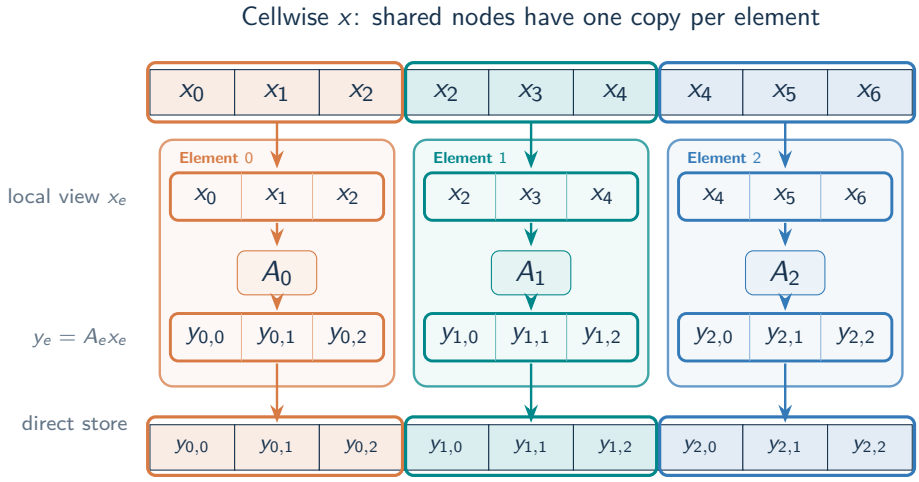
Bottlenecks of an element-parallel global apply



- **Shared writes.** Both elements update the same $y[g]$. Parallel scatter-add needs atomics, coloring, or a reduction.
- **Separate output.** In-place $x \leftarrow Ax$ corrupts pending reads, and scatter-add would include the old x. Accumulate into a distinct $y = 0$.
- **SIMD across elements.** The baseline processes one element at a time. Batching global values needs indexed accesses or repacking.

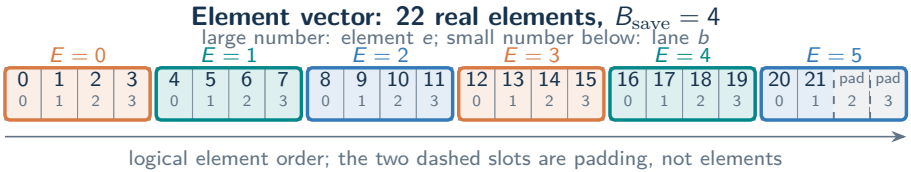
Structured node indices can be implicit; shared output writes remain.

Cellwise apply: element-local copies



Logical element view; BXYZE interleaves lanes. DSS later sums shared outputs.

BXYZE: explicit offsets and padded blocks



Logical index order, slowest to fastest

$$u[E][z][y][x][b]$$

E: storage block; b: lane.

$e = EB_{\text{save}} + b$.

x, y, z: local nodal indices, $N = p + 1$.

Explicit scalar offset

$v = x + N(y + Nz)$ (local node).

$\text{off}(E, v, b) = (EN^3 + v)B_{\text{save}} + b$.

Fixed (E, v): $\text{off}(b + 1) = \text{off}(b) + 1$.

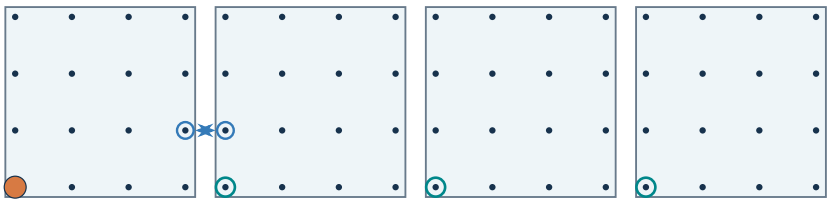
Lanes of one block are contiguous for SIMD.

What padding means

Each block stores $N^3 B_{\text{save}}$ values. In $E = 5$, lanes $b = 2, 3$ have no element and are zero-filled by conversion. The offset formula remains valid. The current affine apply requires full blocks.

BXYZE: b is the fastest index (1/4)

Four $p = 3$ elements at fixed $z = 0$; blue pair = duplicated interface node



Fixed local node $(x, y) = (0, 0)$, so $v = x + 4y = 0$

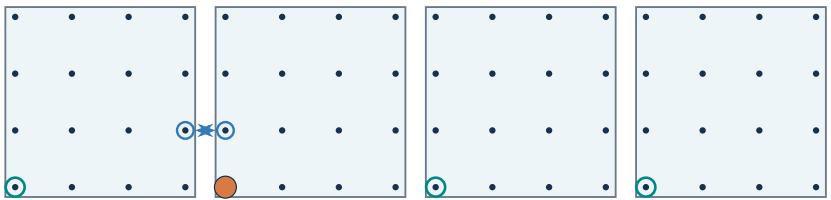
$u_{e_0,0}$ offset 0	$u_{e_1,0}$ offset 1	$u_{e_2,0}$ offset 2	$u_{e_3,0}$ offset 3

$\mathrm{off}(E = 0, v = 0, b) = b$ active lane $b = 0$, offset 0

Only b changes: the highlighted node moves to the next element, and its memory address increases by one.

BXYZE: b is the fastest index (2/4)

Four $p = 3$ elements at fixed $z = 0$; blue pair = duplicated interface node



Fixed local node $(x, y) = (0, 0)$, so $v = x + 4y = 0$

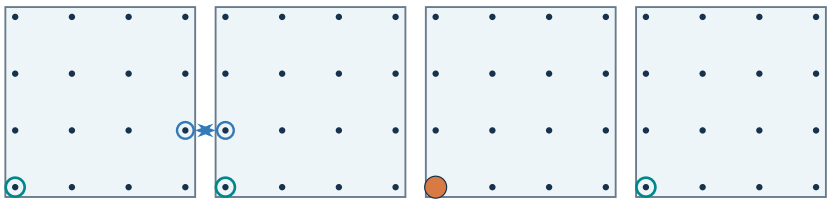
$u_{e_0,0}$ offset 0	$u_{e_1,0}$ offset 1	$u_{e_2,0}$ offset 2	$u_{e_3,0}$ offset 3

$\mathrm{off}(E = 0, v = 0, b) = b$ active lane $b = 1$, offset 1

Only b changes: the highlighted node moves to the next element, and its memory address increases by one.

BXYZE: b is the fastest index (3/4)

Four $p = 3$ elements at fixed $z = 0$; blue pair = duplicated interface node



Fixed local node $(x, y) = (0, 0)$, so $v = x + 4y = 0$

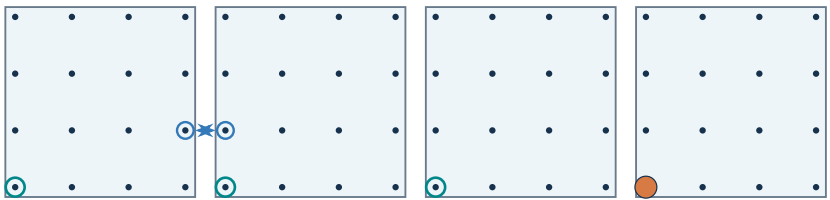
$u_{e_0,0}$ offset 0	$u_{e_1,0}$ offset 1	$u_{e_2,0}$ offset 2	$u_{e_3,0}$ offset 3

$\mathrm{off}(E = 0, v = 0, b) = b$ active lane $b = 2$, offset 2

Only b changes: the highlighted node moves to the next element, and its memory address increases by one.

BXYZE: b is the fastest index (4/4)

Four $p = 3$ elements at fixed $z = 0$; blue pair = duplicated interface node



Fixed local node $(x, y) = (0, 0)$, so $v = x + 4y = 0$

$u_{e_0,0}$ offset 0	$u_{e_1,0}$ offset 1	$u_{e_2,0}$ offset 2	$u_{e_3,0}$ offset 3

$\mathrm{off}(E = 0, v = 0, b) = b$ active lane $b = 3$, offset 3

Only b changes: the highlighted node moves to the next element, and its memory address increases by one.

Cellwise: benefits and costs

Benefits

- **SIMD:** lanes for the same local node are contiguous.
- **No write conflicts:** the local apply needs no atomics or coloring.

Costs

- Shared-node contributions still need summation (DSS).
- Duplicated nodal values increase memory per cellwise vector.

Extra nodal storage for 64^3 elements

$N_{\text{global}} = (64p + 1)^3, \quad N_{\text{cell}} = 64^3(p + 1)^3.$

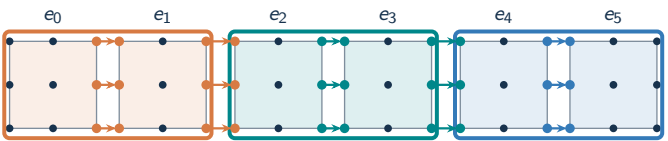
p	Global nodes	Cellwise entries	Extra copies	Extra FP64 memory
2	2 146 689	7 077 888	4 931 199	37.6 MiB
4	16 974 593	32 768 000	15 793 407	120.5 MiB
8	135 005 697	191 102 976	56 097 279	428.0 MiB

At 128^3 elements, $p = 8$: 451 933 183 extra entries $\Rightarrow$ **3.37 GiB** per FP64 vector.

Assumes full blocks; geometry and workspace excluded.

Structured DSS: each element meets its neighbor

One-row Q_2 mesh: $n_x = 6$, $B_{\text{save}} = 2$



Separate squares expose the two copies to sum; arrows use the left chunk's color.

In 3D: three ordered passes

$$z \longrightarrow y \longrightarrow x$$

At each interface, sum matching face-node copies and write the sum to both elements.

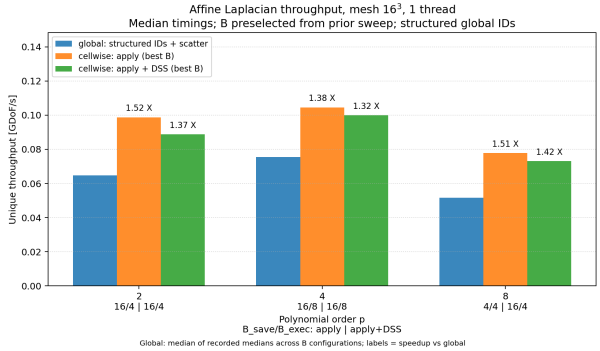
Numbering gives the neighbor directly

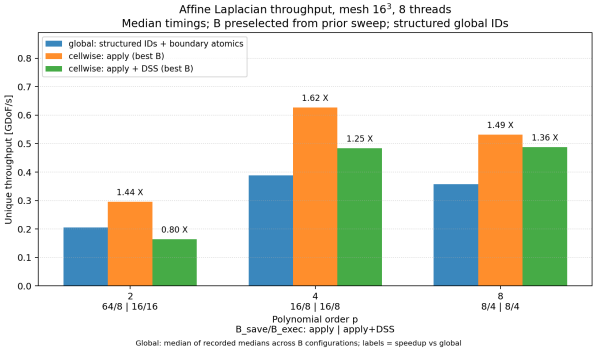
$e = e_x + n_x(e_y + n_y e_z)$.

Given e, add 1 in x, n_x in y, or $n_x n_y$ in z to find the next element. No connectivity lookup.

The $z \to y \to x$ cascade also assembles edge and corner nodes; an OpenMP barrier separates the passes.

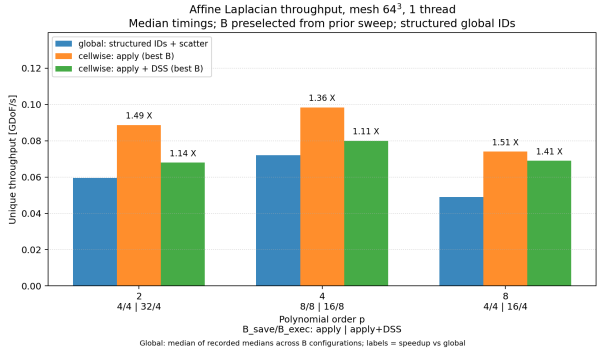
Operator throughput: 16^3 elements

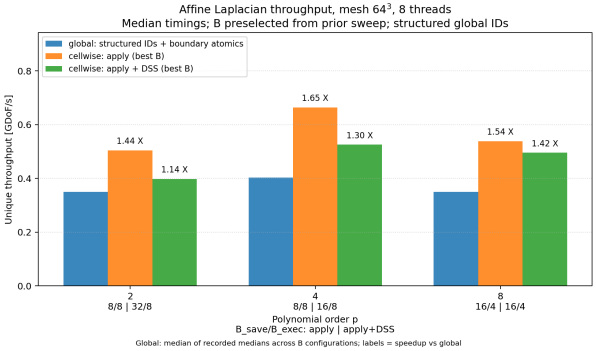




Throughput in unique global DoFs/s; speedups are relative to the global apply.

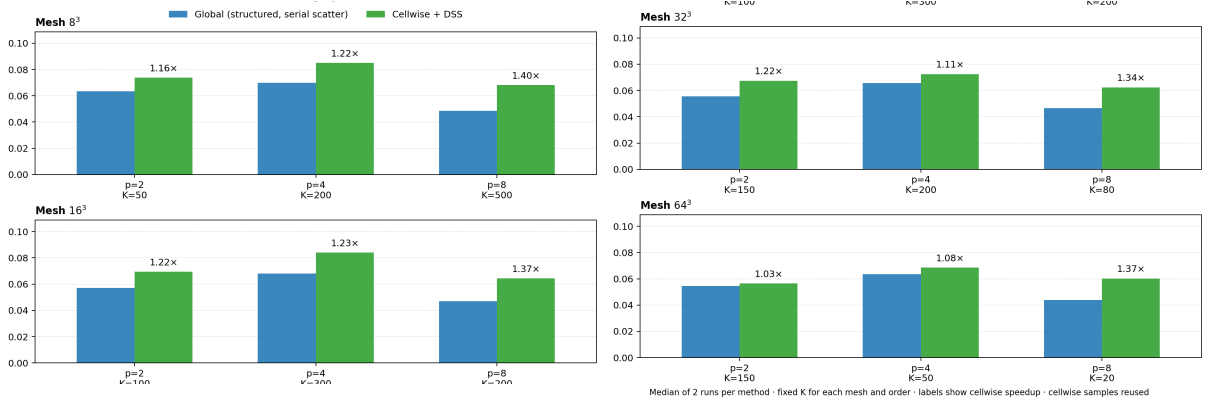
Operator throughput: 64^3 elements





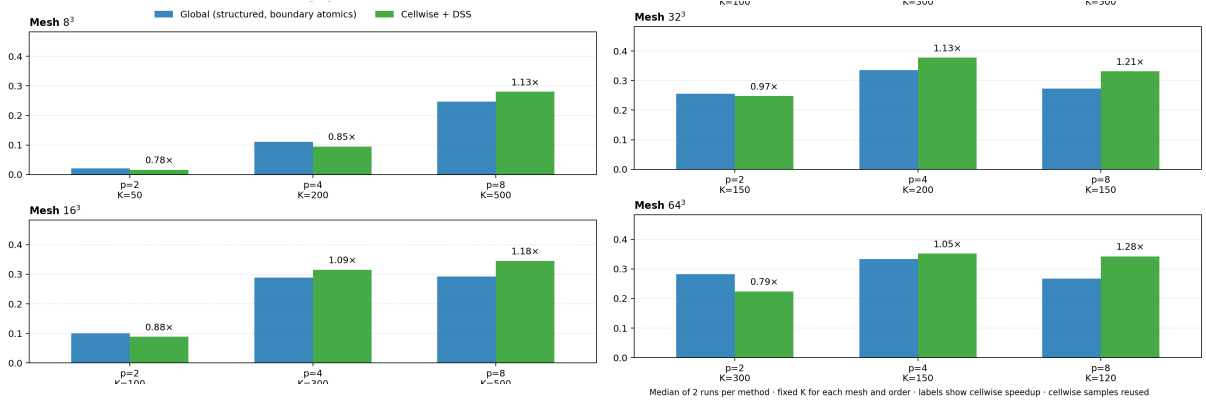
Throughput in unique global DoFs/s; speedups are relative to the global apply.

CG throughput: 1 thread



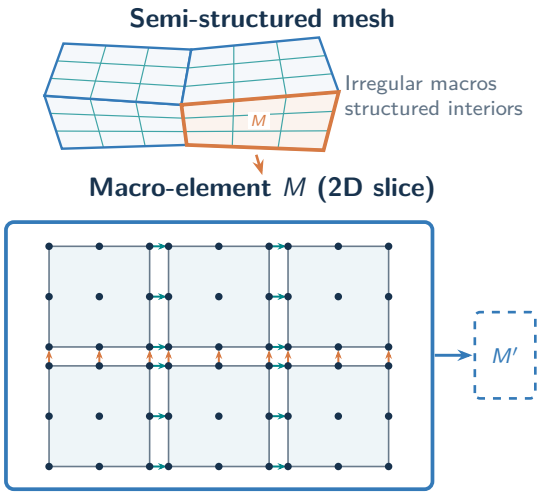
Throughput = unique global DoFs $\times$ K/solve time; K is fixed per case.

CG throughput: 8 threads



Throughput = unique global DoFs $\times$ K/solve time; K is fixed per case.

Next work: semi-structured meshes on GPU



Cellwise copies; arrows show internal DSS.

Within each macro-element

1. **Gather** working values into GPU registers.
2. **Apply** the batched BXYZE cellwise operator.
3. **DSS** the duplicated nodes across structured subelements.
4. **Scatter** to distinct cellwise slots, without atomics inside M.

Between macro-elements

Inter-macro pass algorithm (*Patrick's format*) to assemble the remaining shared interface nodes.